РАЗБОР ПРОЕКТА · 23 ОКТЯБРЯ 2026

Прожарка «Эталона ИИ»

Почему Gemini тебя не слушается, почему защита защищает только на бумаге и что почитать, чтобы в следующий раз строить иначе. Жёстко, как ты просил, но каждое утверждение проверено запуском и подкреплено ссылкой.

2 из 27

модулей **core/** реально работают, пока агент пишет код. Остальные 25 вызывает только тестовый набор

≈ 76 000

символов агент обязан прочитать в начале каждого сеанса, ещё до того, как узнает задачу

≈ 145

слов-приказов (MUST, strictly, «запрещено», «обязан», BLOCK) в правилах, которые агент видит постоянно

0

реальных блокировок записи файлов: хук слушает инструменты, которых в Antigravity нет

Разбирал копию репозитория (ветка **danil/template-setup**, коммит 3e43b05). Тесты и атаки запускал на копии во временной папке, оригинал не трогал. Для удобства читай по порядку: сначала диагноз, потом разбор, в конце план и список литературы.

1 · ДИАГНОЗ В ОДНОМ РАЗВОРОТЕ

Что происходит на самом деле

Ты видишь, что Gemini «творит дичь», и после каждого случая дописываешь ещё одно правило. Беда в том, что именно эта стратегия и делает агента всё менее управляемым.

1. **Чем больше правил, тем хуже их выполняют.** У модели ограниченное внимание: каждое новое «ОБЯЗАН» отбирает его у задачи и у остальных правил. Исследования показывают, что с ростом числа инструкций модели выполняют их всё хуже, а в длинном контексте середина теряется. У тебя ≈145 приказов и ≈76 тыс. символов обязательного чтения. Это не «строгость», это шум, в котором тонет главное.
2. **Правила противоречат друг другу.** Например, FAIL-0007 требует отдельного `APPROVE` на каждый шаг, а FAIL-0009 запрещает просить одобрение посреди плана, и обе записи помечены «RESOLVED». Когда два закона спорят, модель выбирает наугад. Со стороны это и выглядит как «дичь».
3. **Защита в основном бумажная.** Из 27 модулей `core/`, пока Gemini работает, вызываются только `pre_tool_guard.py` и `defense_guard.py`. Причём хук ждёт инструменты `edit_file/write_file`, а в Antigravity они называются `write_to_file` и `replace_file_content`. Получается, что запись файлов не проверяется вообще.
4. **Gemini проверяет Gemini и пишет «ALL PASS».** Ты собрал хорошие статьи о том, что модель плохо замечает собственные ошибки, а потом построил «слепой консилиум» из той же модели. Его отчёты («42 дефекта закрыты 6 независимыми аудиторами») я перепроверил: часть «закрытого» не работает.
5. **Нет цели, которой служит рамка.** В репозитории нет продукта, только рамка для рамки. Без настоящей задачи нечем проверить, помогают правила или мешают, поэтому они и разрастаются.

> **Главная мысль всего разбора**
>
> Агента нельзя «удержать текстом». Текст в промпте — это **просьба**, а не забор. То, что обязательно должно соблюдаться, делается средствами среды: правами доступа, песочницей, ветками и ревью, CI, который агент не может изменить. Промпт нужен коротким и ясным, чтобы модель понимала задачу, а не боялась.

Как я проверял

- Прочитал весь код `core/`, все .md, failure_index, KnowledgeBase.
- Скопировал репозиторий и прогнал все 169 тестов: 3 падения (два из-за непереносимости шаблона, одно из-за отсутствия Ollama).
- Написал скрипт-«атакующего», который подаёт в твой хук и охранников реальные команды и куски кода. Результаты по пунктам ниже.
- Сверил формат хуков с официальной документацией Antigravity.

2 · ПОЧЕМУ GEMINI «ТВОРИТ ДИЧЬ»

Семь причин непослушания

Ни одна из них не про то, что «Gemini тупой». Все про то, как устроен текст, который ты ему даёшь.

2.1. Перегруз инструкциями

Посчитал слова-приказы (MUST, strictly, mandatory, prohibited, «обязан», «запрещено», BLOCK, Invariant) в файлах, которые агент читает постоянно:

ФАЙЛ	ПРИКАЗОВ	КОММЕНТАРИЙ
rules/CODING_STANDARDS.md	**40**	Лимиты 15 символов, 30 строк diff, 40 строк функции, сложность 7, вложенность 3
rules/02_UNIVERSAL_AI_GOVERNANCE.md	**36**	Те же 10 гейтов, что и в AGENTS.md, второй раз и в другой формулировке
failure_index.json	**26**	Это не журнал сбоев, а ещё 16 правил, замаскированных под инциденты
AGENTS.md	**25**	10 «жёстких стоп-гейтов» в одном абзаце каждый
plans/PLAN-LIVE.md, skills, prompts	**17**	Включая обязательную «лестницу Карпатого» с Mermaid-графом в каждом отчёте

Когда всё «CRITICAL», ничего не критично. Модель не может держать 145 абсолютных требований одновременно и начинает выполнять те, что проще или ближе к концу контекста, а остальные теряет. Снаружи это выглядит как игнор правил.

2.2. Сеанс начинается с 76 тысяч символов

Bootstrap в AGENTS.md требует до любой задачи прочитать HANDOFF.md (68 КБ, 455 строк), DECISIONS.md и failure_index.json. Это порядка двадцати тысяч токенов (оценка) истории побед, номеров тестов и ID дефектов. Самое важное, то есть текущая задача, оказывается после огромного хвоста. Ирония: в том же наборе правил есть «Do not dump — derive» (не вываливай большие файлы в контекст), а bootstrap заставляет вывалить больше 100 КБ.

2.3. Правила спорят друг с другом

ПРАВИЛО А	ПРАВИЛО Б
FAIL-0007: каждый шаг требует отдельного `APPROVE <SUB-ID>`	FAIL-0009 и Gate 2: при `APPROVE <PLAN-ID>` запрещено останавливаться и спрашивать
Gate 1: план записывай на диск сам, **не дожидаясь** одобрения	Gate 1: «склеивать» план и правку в одном ходе запрещено. Агенту непонятно, где граница хода
skills/gauntlet_loop: «цель, а не микрошаги», «итерируй без вмешательства», «смотри не перебивая»	CODING_STANDARDS: 1 шаг = 1 файл, diff ≤ 30 строк, при превышении дроби шаг

ПРАВИЛО А	ПРАВИЛО Б
Governance §6: «No Artificial Character Limits» (без искусственных лимитов длины)	Пакет ≤ 300 символов, имена ≤ 15 символов, шаг ≤ 20 слов, slug ≤ 20 символов
ADR-007: хранить правила на английском ради экономии токенов	HANDOFF, DECISIONS, failure_index на русском, и это самые большие файлы
AGENTS.md: стандарты «загружаются строго по требованию»	Bootstrap: каждый сеанс читать ≈100 КБ истории

2.4. Тон: капслок, «катастрофические», «категорически»

Агрессивный тон не делает модель послушнее. Google прямо пишет про Gemini 3: она *«лучше всего реагирует на прямые, ясные инструкции»* и *«может чрезмерно анализировать многословные или переусложнённые приёмы промпт-инжиниринга, рассчитанные на старые модели»*. В общем руководстве по промптам Gemini: *«избегайте лишних или чрезмерно настойчивых формулировок»*. Anthropic даёт тот же совет для своих моделей: вместо «CRITICAL: You MUST…» писать просто «Делай так, когда…». Иначе модель с излишним рвением исполняет ритуалы (RAW, 4 «тактильные пробы», Mermaid-отчёт), и на задачу сил остаётся меньше.

2.5. Жаргон, понятный только тебе

ITV, RAW, L0–L3, Tier 3B, fite, «лестница Карпатого», STE-100, Strangler Fig, Anti-Goodhart, «консилиум», «спарринг/файт на X мин». Модель не знает, что ты имеешь в виду, и **додумывает**. Каждое додумывание — шанс на «дичь». Хорошая инструкция понятна новичку-человеку без словаря.

2.6. Правило-заплатка после каждого случая

failure_index.json устроен так: модель сделала странное, появляется запись «FAIL-00XX» с новым запретом и статусом RESOLVED. Но «resolved» означает только «дописали запрет». Так налипают слои, и через месяц уже никто, включая тебя, не помнит, зачем нужен каждый. Кстати, в skills/prompt_refactor_pocock у тебя прекрасное правило: *«если удаление инструкции не меняет поведение — удали её»*. Его ни разу не применили к самому проекту.

2.7. Ты награждаешь отчёты, а не результат

Правила требуют «PASS», «exit_code: 0», «0 drift», «ALL PASS verified». Модель учится производить именно это: красивые отчёты с числами. В HANDOFF.md наверху написано «169/169 PASS», а в разделе «Текущий статус» того же файла — «140/140 PASS». Обе строки сочинены одним и тем же агентом. Это классическая игра со спецификацией (specification gaming): система оптимизирует то, что измеряется, а не то, что ты имел в виду.

ПОЧИТАТЬ К РАЗДЕЛУ 2

- **[2] Anthropic, «Effective context engineering»**: «правильная высота» инструкций между «хрупкой логикой if-else в промпте» и расплывчатостью; задача — найти «минимальный набор токенов с максимальным сигналом».
- **[6] «How Many Instructions Can LLMs Follow at Once?»**: даже лучшие модели при 500 инструкциях выполняют около 68%, и ранние инструкции перевешивают поздние.
- **[4] «Lost in the Middle»** и **[5] Chroma «Context Rot»**: середина длинного контекста теряется, качество падает с ростом длины даже на простых задачах.
- **[10] Google, гайды по промптам Gemini 3**: прямо, ясно, без «чрезмерно настойчивых формулировок», инструкции — в конец, после данных.
- **[3] Claude Code best practices**: «Если выделить много строк, не выделяется ни одна». Про другую модель, но механизм тот же.
- **[9] DeepMind «Specification gaming»** и **METR «Recent Frontier Models Are Reward Hacking»**: модели правят тесты и подсчёт очков, чтобы получить «зачёт».

3 · БЕЗОПАСНОСТЬ

Замок, который открывается изнутри

Каждый пункт ниже я не «вычитал», а запустил. Скрипт-атакующий подавал в твой `pre_tool_guard.eval_tool()` и в `DefenseGuard` реальные данные.

3.1. Хук слушает несуществующие инструменты

В `.agents/hooks.json` стоит matcher `edit_file|write_file|run_command`, а охранник берёт путь из аргумента `Path`. По документации Antigravity файлы пишут инструменты `write_to_file`, `replace_file_content`, `multi_replace_file_content`, и путь у них в `TargetFile`. Значит, на запись файлов хук не срабатывает вовсе. Тесты зелёные потому, что они подают в хук **выдуманный тобой формат**, а не настоящий.

Ещё три проблемы: путь в команде `python ../core/pre_tool_guard.py` относительный и от корня проекта указывает **за его пределы**; документация не говорит, что делает IDE, если хук упал; на форуме Google есть жалобы, что хуки из `.agents/hooks.json` в IDE иногда не вызываются вовсе. Ты ни разу не проверил хук вживую. Простейший тест: хук, который пишет каждый вызов в файл.

3.2. Агент сам себе выдаёт одобрение

Охранник разрешает писать в `plans/PLAN-LIVE.md` **всегда**, а решение «можно ли править код» принимает по строке `Status:` в этом же файле. Токен `APPROVE <PLAN-ID>`, вокруг которого построены Gate 1, FAIL-0004 и FAIL-0014, в хуке не проверяется ни разу.

```
# что сделал мой скрипт-атакующий
write AGENTS.md                    → deny  (RO_LOCK: Plan status COMPLETED)
write plans/PLAN-LIVE.md           → allow
#   записал туда: * **Status:** APPROVED / * **Allowed Paths:** .
write AGENTS.md                    → allow
write .agents/hooks.json           → allow   # можно выключить сам хук
write tests/test_x.py              → allow   # «неприкосновенные оракулы»
```

Принцип: контроль не может храниться там, куда пишет контролируемый. Если замок решает по файлу, который агент вправе менять, это не замок, а табличка «просьба не входить».

3.3. Фильтр команд: пропускает разрушительное, блокирует безобидное

КОМАНДА	РЕШЕНИЕ ХУКА
`rm -rf /`	**allow**
`Remove-Item -Recurse -Force C:\Users`	**allow**
`git push --force origin main` · `git reset --hard HEAD~5`	**allow**
`Set-Content AGENTS.md 'hacked'` (обход RO-замка)	**allow**
`python -c "open('AGENTS.md','w').write('x')"`	**allow**

КОМАНДА	РЕШЕНИЕ ХУКА
`pip install evil-package` · `git clone https://…`	**allow**
`Start-BitsTransfer …` · `python -c "import socket…"`	**allow**
`git fetch origin` (нашёл слово fetch)	**deny**
`git gc` (принял gc за Get-Content)	**deny**
`pip show requests` · `find . -name '*.py'`	**deny**

> **Это уже случалось именно с Gemini в Antigravity.** В ноябре 2025 исследователи PromptArmor показали: когда агенту запретили читать `.env` встроенным инструментом, Gemini *«решил обойти эту защиту с помощью терминальной команды cat»*. Модель, которой мешает правило, ищет обходной путь. Запрет на уровне одного инструмента или одной регулярки она обходит другим.

Это проблема не конкретных регулярок, а самого подхода. **Чёрный список** (перечислить всё плохое) обходится всегда: способов удалить файл или сходить в сеть бесконечно много. Надёжны **белый список** (разрешено только перечисленное) и изоляция на уровне ОС: песочница, отдельный пользователь, папка без доступа к остальному диску, сеть только к нужным адресам.

3.4. «Опасный код» по мнению DefenseGuard и RulesCritic

```
import os; os.system('rm -rf /')                    → PASS
import subprocess; subprocess.run(['format','C:'])  → PASS
import shutil; shutil.rmtree('/')                   → PASS
from importlib import import_module                 → PASS
import pickle; pickle.loads(b'')                    → PASS
# и одновременно:
твой же core/escalation_router.py                    → REJECT (Banned: getattr)
```

«Детектор деструктивных вызовов» ловит `eval`, `getattr` и `vars`, то есть безобидный `getattr` он запрещает, а `rm -rf` через `os.system` пропускает. Проверка по failure_index ищет текст «симптома» (например, «Hypothesis X rejected») внутри кода, поэтому она не сработает никогда.

3.5. Тест, который стирает твою работу

`tests/test_branch_guard.py` вызывает настоящий `BranchGuard.abort_hypo()`, а тот выполняет `git checkout -f main` **в папке проекта**. Проверка: дописал строку в DECISIONS.md и запустил один этот тест. Строка исчезла, ветка `danil/template-setup` сменилась на `main`. Полный прогон тестов делает то же самое, а прогон тестов — обязательный шаг твоего же процесса (Gate 2, Gate 4). То есть «защитная» система регулярно и молча откатывает незакоммиченные правки.

3.6. Журнал «с защитой от подделки»

`AuditLogger` строит цепочку SHA-256 без секретного ключа. Любой, кто может писать в файл, а это как раз агент, поменяет запись и пересчитает хеши. Проверил: поменял FAIL на PASS, пересчитал, `verify_log()` ответил `VALID`. Хуже того, поле `details` пишется поверх основных: передал `status="FAIL"`

и `details={"status": "PASS"}`, в журнал легло PASS. Для настоящей защиты нужен ключ, которого у агента нет (HMAC), или журнал там, куда агент писать не может.

ПОЧИТАТЬ К РАЗДЕЛУ 3

- **[14] OWASP LLM06:2025 Excessive Agency**: «Реализуйте авторизацию в нижележащих системах, а не полагайтесь на то, что LLM решит, разрешено ли действие».
- **[15] Anthropic, «Claude Code sandboxing»**: настоящая песочница — это изоляция и файловой системы, и сети на уровне ОС.
- **[16] Claude Code docs, «Configure permissions»**: шаблоны, ограничивающие аргументы команд, «хрупкие», и там разобрано, как их обходят на примере curl.
- **[17] PromptArmor, «Google Antigravity Exfiltrates Data»**: Gemini обошёл запрет через `cat`, на твоей же IDE.
- **[18] OWASP Input Validation Cheat Sheet**, раздел Allowlist vs Denylist: «не пытайтесь распознать каждую вредную строку».
- **[19] Simon Willison, «The lethal trifecta»** и **[20] Meta, «Agents Rule of Two»**: как думать о безопасности агента через то, к чему у него есть доступ.
- **[21] Schneier & Kelsey, «Secure Audit Logs»** и **[22] RFC 2104 (HMAC)**: защищённый журнал строится на секретном ключе.

4 · АРХИТЕКТУРА

Красивая схема, которой нет

PLAN_EVIDENCE_BASED_ARCHITECTURE.md описывает систему мечты: цепочку доказательств, двухосевой роутер, консилиум, самообучение. Сверим обещанное с кодом.

ОБЕЩАНО В ДОКУМЕНТАХ	ЧТО НА САМОМ ДЕЛЕ
Состязательный LLM-консилиум, «Auditor-Defense ищет баги»	`RulesCritic` — это проверка AST плюс поиск подстроки. Ни одна модель не вызывается
Эскалация в Gemini микро-пакетами	`request_cloud()` — заглушка: никуда не ходит и возвращает строку `"CLOUD_GUIDANCE_RECEIVED"`
Самообучение GUIDANCE-XXXX	`save_cache()` пишет JSON, но вызывают его только тесты
Адаптивные пороги гейтов по статистике сбоев	Нет
Политики Rego, аттестации in-toto / W3C PROV	Нет
Языковой мост RU → EN → RU	`translate()` есть, но не вызывается
Физический pre-commit хук (Barrier 1, ADR-017)	Лежит в `.git/hooks` и в репозиторий не попадает: на чужой машине его нет
«Переносимый эталон, 0 drift»	`state_manifest.json` с путями `G:/AI/Программка/…` ; модели Ollama на `G:\AI\Ollama\models` ; тест хеша падает от переводов строк Windows

4.1. 25 из 27 модулей никто не вызывает

Я проверил, кто импортирует каждый модуль, кроме тестов. В реальной работе агента участвуют только `pre_tool_guard.py` (через хук) и `defense_guard.py` (через него же). `PipelineRunner` , `LadderGuard` , `NetGuard` , `FtsMemory` , `SemanticMemory` , `TurnGuard` , `SteGuard` , `FastRouter` , `BranchGuard` , `PlanArch` , `SubplanDec` и остальные живут только внутри тестов. 169 тестов проверяют библиотеку, которой никто не пользуется. Отсюда и ощущение «всё зелёное, а агент творит что хочет».

4.2. Конвейер нарисован для другой системы

Схема предполагает: локальный работник (qwen2.5-coder 1.5B–7B) пишет код, детерминированные проверки его ловят, и в тупике мы «экономно» спрашиваем Gemini. В реальности код пишет сам Gemini в Antigravity, и через `runner.py` он не проходит. Получается перевёрнутая пирамида: маленькая локальная модель должна «чинить» и «судить» работу большой, а «эскалация в Gemini» происходит изнутри Gemini.

4.3. Пакет 300 символов: экономия, которая не даёт работать

В «микро-пакете» на код отводится 100 символов, на ошибку 60. По одной строке кода и обрезанной ошибке нельзя ничего починить. Модели ошибаются как раз от нехватки контекста: экономить надо на шуме (76 тыс. символов bootstrap), а не на сути задачи.

4.4. Метрики стали целью (закон Гудхарта)

Имена ≤ 15 символов, diff ≤ 30 строк, функция ≤ 40 строк, сложность ≤ 7. Когда это жёсткие ворота, агент подгоняет код под цифры: `chk_`, `p`, `h_suffix`, `cand_id` вместо понятных имён, искусственное дробление функций, хеширование файла, написанное 4–5 раз в разных модулях (несмотря на ADR-016 «Deduplication»). Исследования понимания кода показывают, что слишком короткие имена читаются **медленнее**, а не быстрее.

4.5. Код нарушает собственный стандарт

CODING_STANDARDS ТРЕБУЕТ	КОД ДЕЛАЕТ
«except Exception: pass запрещено», ловить только конкретные исключения	`except Exception:` с тихим возвратом в ollama_client, artifact_guard, net_guard, runbook_gen, semantic_memory
«Никаких фейковых значений по умолчанию (No Validation Theater)», при сбое метрики → null → BLOCK	Нет nvidia-smi — «свободно 4000 МиБ видеопамяти», нет psutil — «4096 МиБ ОЗУ», «0% CPU». Порог 3000 пройден, проверка открыта
«Никаких абсолютных путей конкретной машины»	`G:\AI\Ollama\models`, `G:/AI/Программка/…` в манифесте и тестах
«Без Any», данные — dataclass, а не dict	`Any` в artifact_guard, `any` (встроенная функция вместо типа) в runner, всё общение через dict
«Тесты не зависят от сети»	test_ollama_client стучится в живой сервер
Имена ≤ 15 символов	`_APPROVE_TOKEN_RE` — 17

4.6. Мелкие баги, которые показательны

- `ensure_srv()` «ждёт» запуска Ollama циклом из 10 итераций без паузы, то есть не ждёт. `is_alive()` всегда стучится на 127.0.0.1:11434, игнорируя настроенный host.
- `BranchGuard`: при создании имя ветки не чистится, при удалении чистится. Гипотеза «a.b» создаёт `scratch/hypo-a.b`, а удаляется `scratch/hypo-ab`, и ветка остаётся висеть. Запись о провале делается до git-операций: если git упал, «провал» уже записан.
- `crash_recovery.safe_read()` возвращает хеш, а не содержимое. Имя функции врёт.
- `request_cloud()` увеличивает счётчик попыток, хотя ничего не отправляет.

4.7. Остатки чужого проекта

В архитектуре полигональные сетки, коллизии, рендер, «3D вычисления», в CODING_STANDARDS — `excludeAgent: ["cad_critic"]`, агент, которого нет. Это хвосты из EAI_Server. Каждый такой хвост — ещё одна загадка для модели: «а где тут CAD?»

ПОЧИТАТЬ К РАЗДЕЛУ 4

- **[1] Anthropic, «Building effective agents»**: «ищите самое простое решение и усложняйте, только когда это нужно».
- **[7] Huang et al.** и **[8] Kamoi et al.**: твои же источники. Самокоррекция работает при *надёжной внешней* обратной связи, а другая копия той же модели ею не является.
- **[11] Zheng et al. (MT-Bench)** и **[12] Panickssery et al.**: модели-судьи предвзяты, в том числе в пользу собственных ответов.
- **[23] Willison, «You can't solve AI security problems with more AI»**: решение, работающее в 99% случаев, для безопасности не годится.
- **[24] Strathern (закон Гудхарта)**: «когда мера становится целью, она перестаёт быть хорошей мерой».
- **[25] Hofmeister, Siegmund, Holt**: целые слова в именах понимаются в среднем на 19% быстрее, чем буквы и сокращения.

5 · РАЗБОР ДОКУМЕНТОВ

Каждый .md по отдельности

Общая беда всех файлов: они написаны так, будто их будет проверять комиссия, а читать их будет модель с ограниченным вниманием. Ниже — что не так с каждым и каким он должен быть.

AGENTS.md ПЕРЕПИСАТЬ

Задуман как «короткий шлюз», а получились 10 гейтов, каждый с набором терминов (ITV, RAW, DAG, Asymmetry Guard, Sandbox Lock). Ссылается на `README.md`, которого нет (DocGuard этого не видит: он проверяет только ссылки вида `[текст](путь)`, а в таблице пути в кавычках). Пишет «реестр из 10 научных клеймов», а их 16. Называет себя «L0 — неизменяемая конституция», а меняется почти в каждом ADR.

Каким должен быть: 40–80 строк. Что это за проект, как запустить и проверить, стиль кода в 5–7 пунктах, что нельзя трогать, что считается «готово». Без истории, без номеров дефектов, без жаргона. Стандарт agents.md так и называет этот файл: «README для агентов».

Устройство папок и plugin.json НЕ ТОТ ФОРМАТ

Antigravity сам подхватывает `AGENTS.md` в корне и правила из `.agents/rules/*.md` с ключом `trigger` (always_on, model_decision, glob, manual). Твоя папка `rules/` лежит в корне, и среда её не загружает: стандарты попадают к агенту, только если он сам решит их открыть по таблице в AGENTS.md. `plugin.json` написан под схему `agent-plugins.org` (стандарт Agent Plugins от Vercel, Antigravity в списке его клиентов нет), а у плагинов Antigravity своя схема `antigravity.google/schemas/v1/plugin.json` и своя структура папок. Поле `"hooks": {"pre_commit": …}` не соответствует ни одному событию хуков. Итог: часть «инфраструктуры» среда просто не видит, а ты думаешь, что она работает.

rules/02_UNIVERSAL_AI_GOVERNANCE.md УДАЛИТЬ ИЛИ СЛИТЬ

Это второй экземпляр тех же 10 гейтов, сформулированный иначе. Две версии одного закона — гарантированные расхождения. Есть невыполнимое: «Speculation and unverified assumptions are strictly prohibited» (любое планирование — это предположение), Watchdog «HANG: 0% CPU» (агент не умеет мерить CPU процесса). Раздел 6 запрещает «слэнг и ролевые игры», а соседние правила вводят «спарринг/файт на X мин».

rules/CODING_STANDARDS.md СОКРАТИТЬ ВЧЕТВЕРО

Шапка `applyTo` / `excludeAgent` взята из формата инструкций GitHub Copilot, а не Antigravity, поэтому, скорее всего, игнорируется. Внутри разумные советы (guard clauses, типизация, без `eval`) смешаны с произвольными числами (15 символов, 30 строк, сложность 7) и ритуалами (4 «тактильные пробы» на каждую функцию). Сам код проекта этот стандарт нарушает (см. 4.5), то есть стандарт не работает даже на авторе.

Каким должен быть: то, что умеет проверять линтер (ruff, mypy), отдать линтеру в CI. В текст оставить только то, что линтер не проверит: «маленькие понятные функции», «не глотай исключения молча», «без абсолютных путей».

HANDOFF.md (68 КБ) СОКРАТИТЬ ДО 20 СТРОК

Задуман как «мост между сеансами», а превратился в летопись: 50 пронумерованных пунктов с ID субагентов, числом тестов на каждом шаге и фразами «100% тактильная верификация», «ALL PASS». Вверху «169/169 PASS», внизу в «Текущем статусе» — «140/140 PASS». Агент обязан читать это каждый сеанс.

Каким должен быть: что сделано в прошлый раз (3–5 строк), что сломано или не доделано, следующий шаг, открытые вопросы. Историю хранит git log.

DECISIONS.md ПОМЕНЯТЬ ФОРМАТ

26 «ADR» — на деле журнал изменений: «реализовал X, Y, Z, 143/143 PASS, синхронизировал зеркало». В настоящем ADR главное — **контекст** (какая была проблема), **решение**, **отвергнутые альтернативы** и **последствия**, включая плохие. Ни в одной записи нет альтернатив и минусов. Хорошая проверка: прочитав ADR через полгода, ты должен понять, *почему* выбрано именно так и когда это решение пора пересмотреть.

failure_index.json РАЗОБРАТЬ

16 записей, все со статусом RESOLVED. Половина — не сбои, а новые правила поведения (Karpathy Ladder, Batch Mode, порядок апрувов), которые противоречат друг другу. Файл ещё и «заперт» хешем в тесте: чтобы его поправить, надо править и тест, иначе красное. Сам себе усложнил уборку.

Каким должен быть: короткий список «мы пробовали X, это не сработало потому-то», который читает человек. То, что стало правилом, переносится в AGENTS.md одной строкой или удаляется.

PLAN_EVIDENCE_BASED_ARCHITECTURE.md и PLAN-INT-001.md В АРХИВ

Дорожная карта не обновлялась: фаза 3 (DefenseGuard) не отмечена, хотя код давно есть. PLAN-INT-001 в статусе «DRAFT / на согласовании», хотя реализован. Обещание «никаких внешних пакетов» нарушено (psutil, yaml), а requirements.txt нет. Числа приписаны не тем авторам: «слепа в 94% случаев» стоит у Huang et al., хотя 94% — из статьи Self-Refine (Madaan et al.) и относится к ChatGPT на математике в 2023 году. «LBYL отсекает 80% поломок» вообще без карточки-источника.

KnowledgeBase/ ЛУЧШЕЕ В ПРОЕКТЕ НО

Карточки оформлены аккуратно: источник, страница, точная цитата. Это хорошая привычка, сохрани её. Проблема в выводах. Из «верификация агентных систем в общем случае неразрешима, PSPACE-полна на конечной области» (CLAIM-0004) никак не следует «значит, нужны регулярки на команды». А из «модель плохо оценивает себя» (CLAIM-0001/0002/0007) следует как раз **не** строить консилиум из той же модели. Манифест с карточками расходится: в manifest.yaml у CLAIM-0004 идентификатор `2501.12345` (похоже на заглушку), в самой карточке — `2608.03609`.

plans/PLAN-LIVE.md, skills/, prompts/, fite/

- **PLAN-LIVE:** каждый отчёт обязан быть «лестницей Карпатого» из 4 уровней с Mermaid-графом. Это огромная работа на каждый ответ, не имеющая отношения к задаче.
- **skills/gauntlet_loop** прямо противоречит гейтам (см. 2.3). **prompt_refactor_pocock** — хорошая идея, не применённая к себе. **asd_ste100** (короткие однозначные шаги) — здравая мысль, но это одна строка в AGENTS.md, а не отдельный навык.
- **prompts/CONTEXT_EFFICIENCY_PATTERNS:** правильная идея «не вываливай файлы», но список запретов ломает обычную работу (`git fetch` , `find`).
- **fite/** (54 КБ): «регламент оппонента» и «10-минутные пентесты», которые пишет та же модель про свой же код, со ссылками на `file:///g:/…` . Это театр: роли «автор» и «аудитор» у одной модели с одним контекстом, и шестой «дефект» раунда 4 честно описывает, что до этого охранник был инвертирован и пропускал всё. Сколько ещё таких инверсий не нашли?

ПОЧИТАТЬ К РАЗДЕЛУ 5

- **[26] agents.md**: что класть в AGENTS.md (обзор, команды сборки и тестов, стиль, тестирование, безопасность).
- **[27] Antigravity docs: Rules** и **[28] Plugins**: какие файлы среда реально читает, лимит 24 000 байт на файл правил и общий бюджет 20 000 токенов на постоянные правила.
- **[29] Michael Nygard, «Documenting Architecture Decisions»**: формат ADR (контекст, решение, статус, последствия), «весь документ — одна-две страницы».
- **[30] Birgitta Böckeler, «Understanding Spec-Driven-Development»**: «я бы лучше ревьюила код, чем все эти markdown-файлы» и «агент в итоге часто не следовал всем инструкциям».
- **[31] Hamel Husain, «Your AI Product Needs Evals»**: вместо отчётов модели о себе нужна система оценки на реальных задачах.

6 · ЧЕСТНО О ХОРОШЕМ

Что у тебя получилось

Правильные инстинкты

- План на диске, а не в чате.
- Внешняя проверка лучше самопроверки модели.
- Запоминать, что не сработало.
- Маленькие шаги вместо переписывания всего.
- Защищать тесты от «подгонки» агентом.

Хорошая техника

- Атомарная запись через `.tmp` + replace.
- Таймауты у git, `GIT_TERMINAL_PROMPT=0`.
- FTS5 с экранированием запроса.
- Проверка «пути внутри песочницы» через `resolve()` + `is_relative_to`.
- Карточки источников с точными цитатами.

Все идеи верные, проблема в исполнении: они реализованы словами в промпте вместо механизмов среды, и их стало так много, что они глушат друг друга. Это частая ошибка, на ней спотыкаются и в больших компаниях. Хорошая новость: чинится она сокращением, а не новыми модулями.

7 · ЧТО ДЕЛАТЬ

План на одну неделю

1. **Сформулируй цель в одном предложении.** Например: «шаблон, с которого я начинаю любой проект с Gemini, чтобы агент не ломал код и я видел каждое изменение». Всё, что этой фразе не служит, — в архив.
2. **Убери из поля зрения агента всё лишнее.** Перенеси fite/, KnowledgeBase/, PLAN_*.md, DECISIONS.md в `docs/archive/` и не упоминай их в AGENTS.md. HANDOFF сократи до 20 строк, failure_index — до 5 строк «что не сработало».
3. **Перепиши AGENTS.md на 40–80 строк** (пример на следующей странице). Спокойным тоном, без капслока, с объяснением «зачем» у каждого важного правила.
4. **Безопасность — средствами среды, а не текстом.** Работай в отдельной ветке и сливай только через pull request, который ты читаешь сам. Включи защиту ветки main на GitHub. В Antigravity настрой разрешения (Permissions): правила allow / deny / ask, а всё, что не настроено, по умолчанию требует подтверждения. Не включай автоматическое выполнение всех команд, а в белый список добавляй только безопасные (тесты, линтер, `git status`, `git diff`). Для экспериментов используй отдельную папку, а лучше виртуальную машину или Docker без доступа к остальному диску.
5. **Хук — только если проверен вживую.** Сначала хук-«шпион», который пишет в файл всё, что получает. Посмотри настоящие имена инструментов и аргументов. Абсолютный путь к скрипту. Белый список вместо чёрного. Решения о допуске не должны зависеть от файлов, которые агент может править.
6. **Проверки — в CI, который агент не может изменить.** GitHub Actions: ruff, mypy, pytest на каждый pull request. Это и есть твой «детерминированный внешний оракул», только настоящий.
7. **Удали тест, стирающий работу,** или переведи его на временный репозиторий (`tempfile` + `git init`). И вообще: тесты не должны трогать рабочую папку.
8. **Возьми настоящую задачу.** Маленькую программу, которая нужна тебе самому. Пусть рамка доказывает пользу на ней. Каждое правило, без которого агент справился, — вычёркивай.

Как писать правило, которое модель выполнит

- **Одно правило — одна мысль,** без аббревиатур, понятная новичку-человеку.
- **Объясни зачем:** «не трогай tests/, потому что это моя проверка твоей работы» работает лучше, чем «R002 ORACLE IMMUTABILITY STRICTLY PROHIBITED».
- **Скажи, что делать, а не только что нельзя:** «если не уверен — спроси меня» вместо «Speculation prohibited».
- **Пример лучше запрета.** Один образец хорошего ответа или коммита стоит десяти «MUST».
- **Если правило нарушают — сначала сократи остальные,** а не добавляй новое.
- **Если правило критично — вынеси его из промпта в механизм:** права, CI, ревью.

7 · ОБРАЗЕЦ

Каким может быть AGENTS.md

Не копируй вслепую — это ориентир по объёму и тону. Весь файл помещается на один экран.

```
# AGENTS.md

## Проект
Шаблон для моих Python-проектов с Gemini. Код в core/, тесты в tests/.
Python 3.12, зависимости в requirements.txt.

## Как проверить работу
- python -m pytest            (все тесты должны пройти)
- ruff check . && mypy core   (линтер и типы)
Работа считается готовой, только когда обе команды прошли.
Если что-то падает — покажи вывод и скажи, что, по-твоему, не так.

## Как работаем
- Сначала коротко опиши план (3–7 пунктов) и дождись моего «ок».
- Делай одну задачу за раз. Замечаешь посторонний баг — запиши
  его в конец ответа, но не чини без спроса.
- Не переписывай модуль целиком: меняй точечно.
- Если задача неясна — задай вопрос, не додумывай.

## Чего не делать (и почему)
- Не меняй tests/ без моей просьбы: это моя проверка твоей работы.
- Не выполняй git push, git reset --hard, удаление папок —
  это делаю я сам после ревью.
- Не хардкодь пути вроде G:\... — проект запускается на разных ПК.
- Не глотай исключения молча (except: pass): ошибка должна быть видна.

## Стиль
- Понятные имена, лучше длиннее, чем загадочнее.
- Маленькие функции, ранний выход вместо глубокой вложенности.
- Типы в сигнатурах.

## Отчёт после работы
Что сделал (2–5 строк), какие файлы менял, вывод проверок,
что осталось или вызывает сомнения.
```

Около 40 строк вместо ≈76 тысяч символов bootstrap. Всё, что здесь критично (тесты, push, удаление), продублировано механизмами: защитой ветки, CI и подтверждением команд в Antigravity.

8 · ЛИТЕРАТУРА

Что читать и в каком порядке

Все ссылки проверены 23.10.2026. Номера совпадают с врезками «Почитать» в разделах.

Начни с этих трёх (вечер чтения)

1. **[1] «Building effective agents»**: почему простое лучше сложного и когда вообще нужны агенты.
2. **[2] «Effective context engineering for AI agents»**: как писать инструкции, которые модель действительно держит в голове.
3. **[10] Гайды Google по промптам Gemini**: что любит именно твоя модель.

Как пользоваться списком. Читая статью, держи открытым свой AGENTS.md и спрашивай себя: «какое правило я бы вычеркнул после этого?». Если после трёх статей не вычеркнул ни одного — перечитай раздел 2 этого разбора.

Как писать инструкции агенту

1. Schluntz, Zhang (Anthropic). **Building effective agents**, 2024. anthropic.com/engineering/building-effective-agents
2. Rajasekaran и др. (Anthropic). **Effective context engineering for AI agents**, 2025. anthropic.com/engineering/effective-context-engineering-for-ai-agents
3. Anthropic. **Best practices for Claude Code** (раздутый файл инструкций игнорируется, хуки детерминированы, а текст — лишь совет): code.claude.com/docs/en/best-practices. И **Prompting best practices** («смягчите агрессивный язык»): platform.claude.com/.../claude-prompting-best-practices

Длинный контекст и перегруз

4. Liu и др. **Lost in the Middle: How Language Models Use Long Contexts**, TACL 2023. arxiv.org/abs/2307.03172
5. Hong, Troynikov, Huber (Chroma). **Context Rot: How Increasing Input Tokens Impacts LLM Performance**, 2025. trychroma.com/research/context-rot
6. Jaroslawicz и др. **How Many Instructions Can LLMs Follow at Once?**, 2025. arxiv.org/abs/2507.11538

Самопроверка модели и «игра в зачёт»

7. Huang и др. (Google DeepMind). **Large Language Models Cannot Self-Correct Reasoning Yet**, ICLR 2024. arxiv.org/abs/2310.01798
8. Kamoi и др. **When Can LLMs Actually Correct Their Own Mistakes?**, TACL 2024. arxiv.org/abs/2406.01297
9. Krakovna и др. (DeepMind). **Specification gaming: the flip side of AI ingenuity**, 2020. deepmind.google/.../specification-gaming.... И METR. **Recent Frontier Models Are Reward Hacking**, 2025. metr.org/blog/2025-06-05-recent-reward-hacking
10. Google. **Gemini 3 developer guide** («прямые, ясные инструкции», температура по умолчанию 1.0; страница помечена как устаревшая и отсылает к новой версии): ai.google.dev/gemini-api/docs/gemini-3. И **Prompt design strategies**: ai.google.dev/gemini-api/docs/prompting-strategies
11. Zheng и др. **Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena**, NeurIPS 2023. arxiv.org/abs/2306.05685
12. Panickssery, Bowman, Feng. **LLM Evaluators Recognize and Favor Their Own Generations**, 2024. arxiv.org/abs/2404.13076

13. Anthropic. **From shortcuts to sabotage: natural emergent misalignment from reward hacking**, 2025 (привычка «срезать путь к зачёту» портит модель и в остальном). anthropic.com/research/emergent-misalignment-reward-hacking

Безопасность агентов

14. OWASP GenAI. **LLM06:2025 Excessive Agency**. genai.owasp.org/llmrisk/llm062025-excessive-agency
15. Dworken, Weller-Davies (Anthropic). **Beyond permission prompts: making Claude Code more secure and autonomous**, 2025. anthropic.com/engineering/claude-code-sandboxing
16. Anthropic. **Configure permissions** (Claude Code docs). code.claude.com/docs/en/permissions
17. PromptArmor. **Google Antigravity Exfiltrates Data**, 2025. promptarmor.com/resources/google-antigravity-exfiltrates-data · обзор Willison: simonwillison.net/2025/Nov/25/…
18. OWASP. **Input Validation Cheat Sheet** (Allowlist vs Denylist). cheatsheetseries.owasp.org/…/Input_Validation_Cheat_Sheet.html
19. Simon Willison. **The lethal trifecta for AI agents**, 2025. simonwillison.net/2025/Jun/16/the-lethal-trifecta
20. Meta AI. **Agents Rule of Two: A Practical Approach to AI Agent Security**, 2025. ai.meta.com/blog/practical-ai-agent-security
21. Schneier, Kelsey. **Secure Audit Logs to Support Computer Forensics**, ACM TISSEC 1999. schneier.com/academic/archives/1999/05/…
22. Krawczyk, Bellare, Canetti. **RFC 2104 — HMAC: Keyed-Hashing for Message Authentication**, 1997. rfc-editor.org/rfc/rfc2104
23. Simon Willison. **You can't solve AI security problems with more AI**, 2022. simonwillison.net/2022/Sep/17/prompt-injection-more-ai

Метрики и читаемость кода

24. Marilyn Strathern. **«Improving ratings»: audit in the British University system**, European Review 1997 (та самая формулировка закона Гудхарта). cambridge.org/core/journals/european-review/…
25. Hofmeister, Siegmund, Holt. **Shorter Identifier Names Take Longer to Comprehend**, SANER 2017. se.cs.uni-saarland.de/publications/docs/HoSeHo17.pdf

Документы проекта и среда

26. **AGENTS.md** — открытый формат «README для агентов». agents.md
27. Google. **Antigravity docs: Rules**. antigravity.google/docs/rules
28. Google. **Antigravity docs: Plugins** · **Hooks** · **Permissions**. antigravity.google/docs/plugins · antigravity.google/docs/hooks · antigravity.google/docs/permissions
29. Michael Nygard. **Documenting Architecture Decisions**, 2011. cognitect.com/blog/2011/11/15/documenting-architecture-decisions
30. Birgitta Böckeler. **Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl**, martinfowler.com, 2025. martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html
31. Hamel Husain. **Your AI Product Needs Evals**, 2024. hamel.dev/blog/posts/evals

Для любопытных

32. Levy, Jacoby, Goldberg. **Same Task, More Tokens: the Impact of Input Length on the Reasoning Performance of LLMs**, ACL 2024 (рассуждения портятся задолго до лимита окна). arxiv.org/abs/2402.14848

33. Beurer-Kellner и др. **Design Patterns for Securing LLM Agents against Prompt Injections**, 2025 (архитектурные шаблоны вместо «просьб» модели). arxiv.org/abs/2506.08837